

DD2380 - Machine Learning an introduction

Josephine Sullivan

October, 2026

What is Machine Learning?

“Machine learning is the science of getting computers to act without being explicitly programmed.”

Andrew Ng via Coursera.

“Learning is any process by which a system improves performance from experience.”

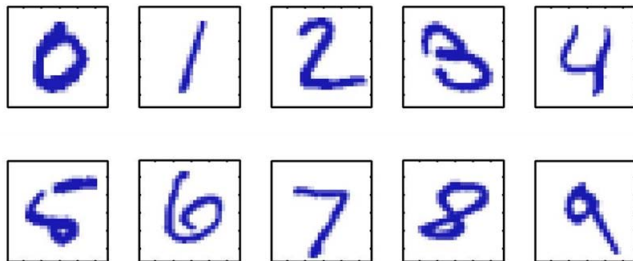
Herbert Simon.

Classic example of a task requiring machine learning



- Can you write down rules to define a chair?
- How would you encode these rules in a computer programme to recognise an image of a chair???

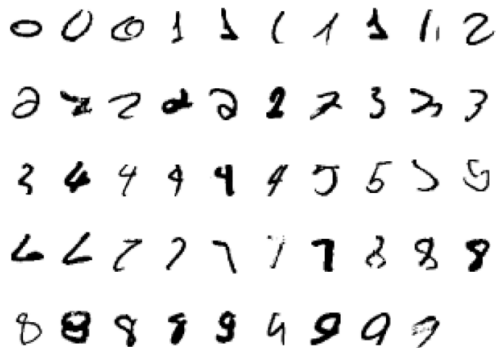
Machine Learning Solution: Hand-written digit recognition



- Images are 28×28 arrays of numbers.
- Represent input image as a vector $\mathbf{x} \in \mathbb{R}^{784}$
- Learn a classifier function s.t.

$$f : \mathbb{R}^{784} \rightarrow \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

- As a supervised classification problem.
- Start with training data

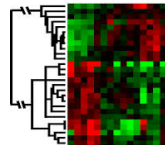


- Can achieve testing error of $\leq 0.4\%$
- One of first commercial and widely used ML systems (for zip codes & cheques)

When is Machine Learning useful?

Use **Machine Learning** when

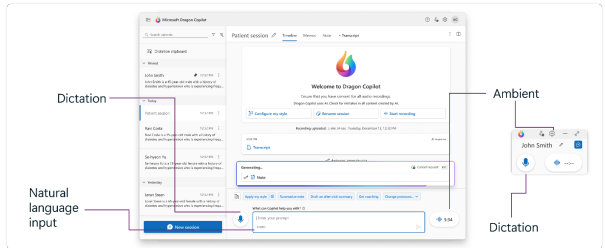
- The rules are difficult to write down
(vision, speech, language, anomaly detection)
- We want to discover structure in data
(clustering, representations, scientific patterns)
- Learn complex relationships between paired data.
- The system needs to adapt or personalize
(recommendations, ranking, personalized services)



Use ML when you can show the computer examples more

Machine Learning is pervasive today

Example: Speech Recognition + NLP + Ambient AI when visiting the doctor



Microsoft Dragon Copilot + others: **Reducing admin burden on doctors**

Where is AI progressing quickly?

- **Foundation models**
language, reasoning, mathematics, coding
- **Multimodal generation**
text, images, speech, music, video
- **Agents & tool use**
browsing, coding, computer use, multi-step tasks
- **AI for science**
proteins, molecules, weather, scientific discovery
- **Robotics / physical AI**
perception, manipulation, autonomous systems

Capabilities are advancing rapidly but unevenly (the jagged frontier)!

In particular last decade has seen rapid progress in generative AI:

Generative Deep Learning at massive scales

Timeline of images generated by AI

Timeline of images generated by artificial intelligence

These people don't exist. All images were generated by artificial intelligence.



2014



Goodfellow et al. (2014) - Generative Adversarial Networks

2015



Balabit, Metti, and Chiraly (2015) - Unsupervised Representation Learning with Deep Convolutional GANs

2016



Li and Teed (2016) - Coupled GANs

2017



Karras et al. (2017) - Progressive Growing of GANs for Improved Quality, Stability, and Variation

2018



Karras, Laine, and Aila (2018) - A Style-Based Generator Architecture for Generative Adversarial Networks

2019



Karras et al. (2019) - Analyzing and Improving the Image Quality of StyleGAN

2020



Ho, Jain, & Abbeel (2020) - Denoising Diffusion Probabilistic Models

2021

Image generated with the prompt: "a couple of people are sitting on a wood bench"



Renesh et al. (2021) - Zero-Shot Text-to-Image Generation (ZeroW-GAN v1.1)

2022

Image generated with the prompt: "A Pomeranian is sitting on the King's throne wearing a crown. Two tiger soldiers are standing next to the throne."



Saharia et al. (2022) - Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding (Google's Imagen)

OurWorldInData.org - Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Charlie Giattino and Max Roser

OpenAI Sora - Movie generation

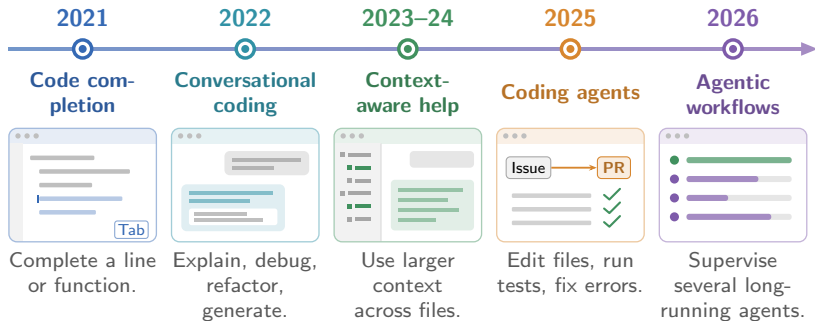
Prompt: A woman wearing a green dress and a sun hat taking a pleasant stroll in Mumbai India during a winter storm.



Sora movie

Movie credit: OpenAI sora - [Creating video from text.](#)

Rapid development of AI-assisted coding

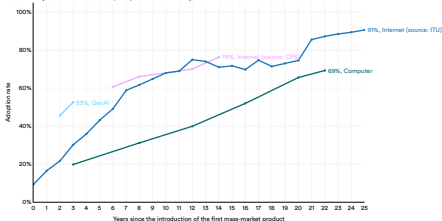


From **predicting the next line of code** to **carrying out software-engineering tasks**.

And it's changing how we work

Speed of AI adoption by technology

Source: The Project on Workforce at Harvard, 2022 | Chart: 2026 AI Index report



- 88% of surveyed organizations use AI
- 70% use generative AI in at least one business function
- Productivity gains can be large; but it depends strongly on the task

Currently AI is changing how tasks are completed rather than replace entire jobs.

But major challenges in ML / AI remain

G

Reliable generalization

Distribution shift, rare cases, and messy real-world deployment.

R

Reliability & uncertainty

Avoid confident errors and know when not to answer.

E

Evaluation

Do our benchmarks measure what actually matters?

I

Interpretability & control

Can we understand, steer, and supervise model behaviour?

S

Specification & security

Is it solving our task—or exploiting shortcuts and loopholes?

D

Data & efficiency

Better data, less compute, privacy, bias, and continual adaptation.

More examples/info about AI developments

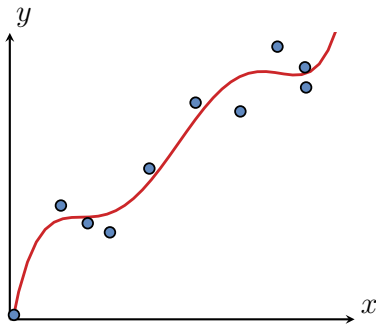
Check out

- [State of AI Report](#), Updated yearly during October.
- [Artificial Intelligence Index Report](#) published yearly by Stanford University, Human-Centered Artificial Intelligence.

Four canonical learning problems

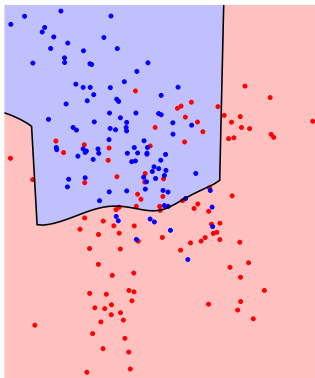
1. Supervised Regression

- Learn a mapping from $\mathbb{R}^d$ to $\mathbb{R}^k$ where $k \geq 1$.
- Have labelled examples for training.



2. Supervised Classification

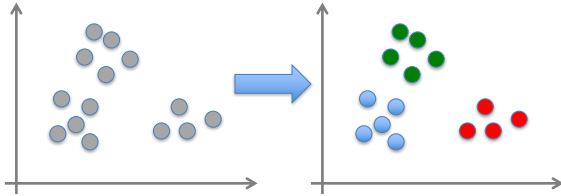
- Learn a mapping from $\mathbb{R}^d$ to $\{1, 2, 3, \dots, k\}$ where $k \geq 2$.
- Have labelled examples for training.



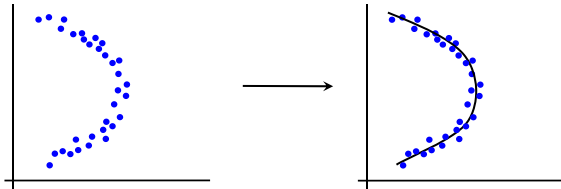
Four canonical learning problems

3. Unsupervised learning - model the data

- Clustering



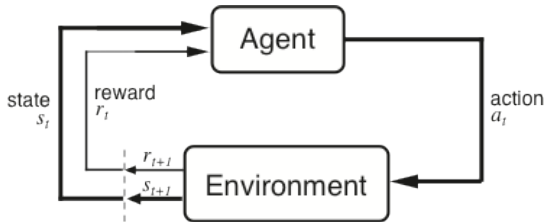
- Dimensionality reduction



Four canonical learning problems

4. Reinforcement learning

- Rewards from sequence of actions



Conventional wisdom on these canonical learning regimes

"If intelligence was a cake, unsupervised learning would be the cake, supervised learning would be the icing on the cake, and reinforcement learning would be the cherry on the cake."

Yann LeCun, pre-2019

New more recent learning paradigm

*"If intelligence was a cake, **self-supervised** learning would be the cake, supervised learning would be the icing on the cake, and reinforcement learning would be the cherry on the cake."*

Yann LeCun, from 2019 onwards.

- Self-supervised learning (SSL) training regime of most foundation models.
- SSL is a form of unsupervised learning where the input data generates its own labels
- But luckily learning/optimization happens as in supervised learning.

New more recent learning paradigm

*"If intelligence was a cake, **self-supervised** learning would be the cake, supervised learning would be the icing on the cake, and reinforcement learning would be the cherry on the cake."*

Yann LeCun, from 2019 onwards.

- Self-supervised learning (SSL) training regime of most foundation models.
- SSL is a form of unsupervised learning where the input data generates its own labels
- But luckily learning/optimization happens as in supervised learning.

Learn to represent the world before learning a task.

- Self-supervised learning for NLP - biggest success story so far.
- Modern LLMs are typically pre-trained using self-supervision

- **Early example of a self-supervised task in NLP:**

- **Unlabelled data:** Collect many valid sentences.

The man went to the shop to buy a pint of milk.

The rain in Spain falls mainly in the plain.

....

- **Task:** Mask out input words in each sentence

The man went to the [MASK] to buy a pint of [MASK].

and train network to predict masked words:

shop milk
↑ ↑
The man went to the [MASK] to buy a pint of [MASK].

- Focus on the problem of supervised regression.
- **Why?** Highlight transferable issues that arise in generic ML.

Devil's advocate: It's all just curve fitting!

*As much as I look into what's being done with deep learning, I see **they're all stuck there on the level of associations. Curve fitting.** That sounds like sacrilege, to say that all the impressive achievements of deep learning amount to just fitting a curve to data. From the point of view of the mathematical hierarchy, **no matter how skillfully you manipulate the data and what you read into the data when you manipulate it, it's still a curve-fitting exercise, albeit complex and non-trivial.***

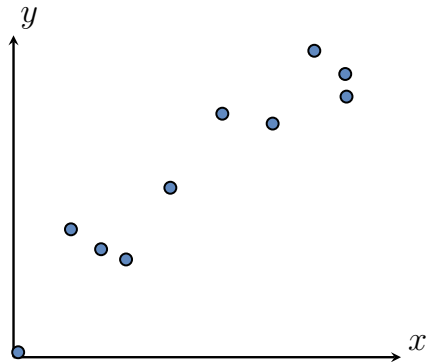
Judea Pearl.

- As soon as AI successfully solves a problem, solving the problem is no longer a demonstration of AI..
- For example it was thought “real” artificial intelligence needed for computers to
 - beat chess Grand masters at chess,
 - beat expert Go players at Go,
 - perform object recognition,
 - perform accurate speech recognition,
 -
- Larry Tesler expresses the **AI effect** as:

“AI is whatever hasn’t been done yet.”

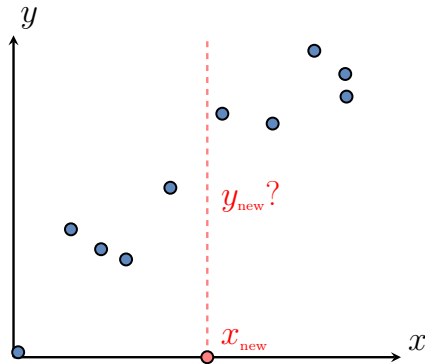
Supervised learning - Regression

Supervised Regression: What you are given



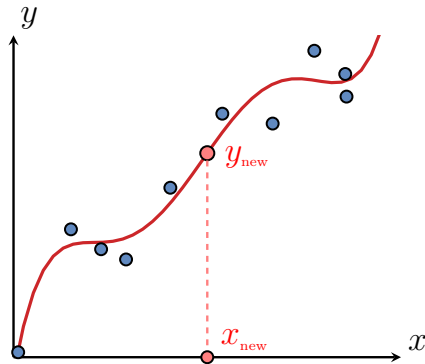
- **Given:** Labelled training data $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^n$ where each $x_i \in \mathbb{R}$ and $y_i \in \mathbb{R}$.

Supervised Regression: The task



- **Task:** for any $x_{\text{new}} \in \mathbb{R}$ predict its y_{new} value.

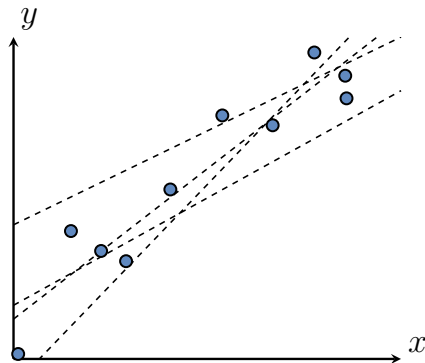
Supervised Regression: High level solution



- **Solution:** Learn a function, $f : \mathbb{R} \rightarrow \mathbb{R}$, that predicts output value y_{new} for input x_{new} that is

$$f(x_{\text{new}}) = y_{\text{new}}$$

Supervised Regression: Learn which function?



Learning requires:

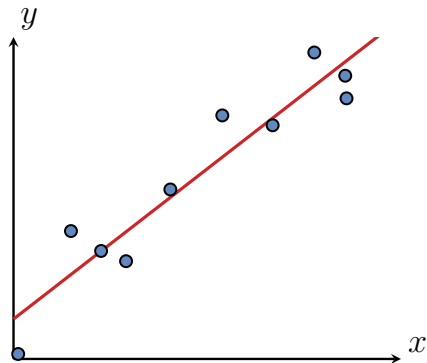
1. Defining type of $f : \mathbb{R} \times \mathbb{R}^p \rightarrow \mathbb{R}$ such as a linear function

$$f(x, \theta) = wx + b \quad \text{where } \theta = \begin{pmatrix} b \\ w \end{pmatrix}$$

with parameters θ controlling its exact shape.

2. Estimating a good θ^* s.t. $f(x_i, \theta^*) \approx y_i$ for $i = 1, \dots, n$

Supervised Regression: Learn which function?



Learning requires:

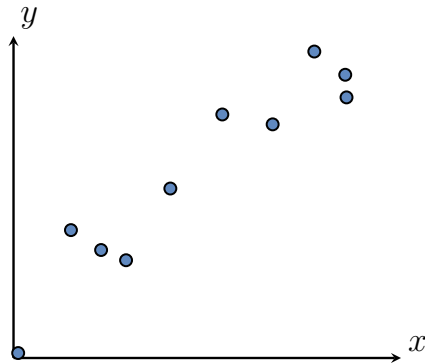
1. Defining type of $f : \mathbb{R} \times \mathbb{R}^p \rightarrow \mathbb{R}$ such as a linear function

$$f(x, \theta) = wx + b \quad \text{where } \theta = \begin{pmatrix} b \\ w \end{pmatrix}$$

with parameters θ controlling its exact shape.

2. Estimating a good θ^* s.t. $f(x_i, \theta^*) \approx y_i$ for $i = 1, \dots, n$

Focus on estimating θ from training data

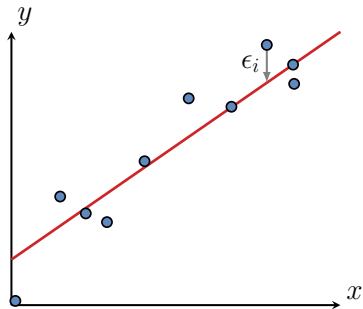


- In this example there appears to be a linear relationship between x and y .
- So let

$$f(x, \theta) = wx + b \quad \text{where } \theta = \begin{pmatrix} b \\ w \end{pmatrix}$$

- How can we estimate a good θ from the training data $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^n$?

Least squares estimation of θ



- How to estimate θ from the training data?

- Want

$$f(x_i, \theta) = wx_i + b \approx y_i \quad \text{for } i = 1, \dots, n$$

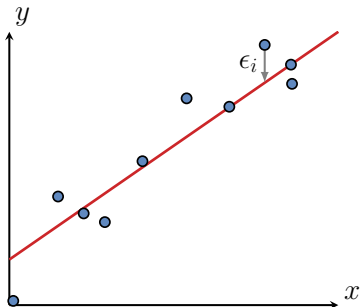
- So set up this least squares optimization problem

$$w^*, b^* = \arg \min_{w, b} \sum_{i=1}^n \epsilon_i^2$$

where

$$\epsilon_i = wx_i + b - y_i$$

Can solve this using calculus!



- Let

$$C(w, b, \mathcal{X}) = \sum_{i=1}^n \epsilon_i^2$$

- Take partial derivatives of C w.r.t. w and b and set them to zero:

$$\frac{\partial C}{\partial w} = 0 \quad \text{and} \quad \frac{\partial C}{\partial b} = 0$$

- Solve this system of equations - two unknowns and two linear constraints.

- Expression for the partial derivatives

$$\frac{\partial C}{\partial w} = 2 \sum_{i=1}^n (wx_i + b - y_i)x_i, \quad \frac{\partial C}{\partial b} = 2 \sum_{i=1}^n (wx_i + b - y_i)$$

- Set these two equations to zero and solve for w and b :

$$w^* = \frac{\sum_i x_i y_i - n \bar{y} \bar{x}}{\sum_i x_i^2 - n \bar{x}^2},$$
$$b^* = \bar{y} - w^* \bar{x}$$

where

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad \text{and} \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

Can write what we have done in matrix notation

- Let

$$X = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad \text{and} \quad \boldsymbol{\theta} = \begin{pmatrix} b \\ w \end{pmatrix}$$

- Then

$$\mathbf{y} = X\boldsymbol{\theta}$$

and

$$C(\boldsymbol{\theta}, \mathcal{X}) = (X\boldsymbol{\theta} - \mathbf{y})^T (X\boldsymbol{\theta} - \mathbf{y})$$

- If we know some vector calculus.....

Can write what we have done in matrix notation

- Let

$$X = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad \text{and} \quad \boldsymbol{\theta} = \begin{pmatrix} b \\ w \end{pmatrix}$$

- Then

$$\mathbf{y} = X\boldsymbol{\theta}$$

and

$$C(\boldsymbol{\theta}, \mathcal{X}) = (X\boldsymbol{\theta} - \mathbf{y})^T (X\boldsymbol{\theta} - \mathbf{y})$$

- If we know some vector calculus.....

Can write what we have done in matrix notation

- Compute the gradient of C w.r.t. θ and set to 0

$$\frac{\partial C}{\partial \theta} = 2X^T X \theta - 2X^T \mathbf{y} = 0$$

"The Matrix Cookbook" is your friend.

- The optimal θ is then given by (if $X^T X$ is invertible)

$$\theta^* = (X^T X)^{-1} X^T \mathbf{y}$$

- Cleaner specification of the solution worth the initial extra effort, especially if we consider....

Can write what we have done in matrix notation

- Compute the gradient of C w.r.t. θ and set to 0

$$\frac{\partial C}{\partial \theta} = 2X^T X \theta - 2X^T \mathbf{y} = 0$$

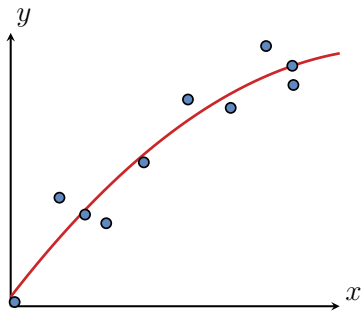
"The Matrix Cookbook" is your friend.

- The optimal θ is then given by (if $X^T X$ is invertible)

$$\theta^* = (X^T X)^{-1} X^T \mathbf{y}$$

- Cleaner specification of the solution worth the initial extra effort, especially if we consider....

Are we really sure f should be linear?



- Perhaps a better model of the training data would be a quadratic function:

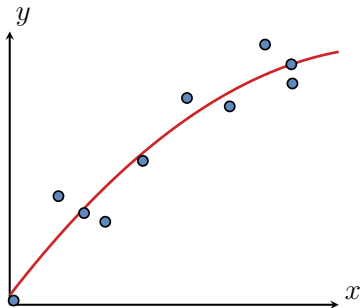
$$f(x, \theta) = w_1 + w_2x + w_3x^2$$

where $\theta = (w_1, w_2, w_3)^T$

- We can find an estimate for θ by solving the same least squares problem as before.

$$\arg \min_{\theta} \sum_{i=1}^n (w_1 + w_2x_i + w_3x_i^2 - y_i)^2$$

Least squares for fitting a non-linear function



- We can find an estimate for θ by solving the same least squares problem as before.

$$\arg \min_{\theta} \sum_{i=1}^n (w_1 + w_2 x_i + w_3 x_i^2 - y_i)^2$$

- Can write this in matrix notation

$$\arg \min_{\theta} (X\theta - \mathbf{y})^T (X\theta - \mathbf{y})$$

where

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \\ 1 & x_n & x_n^2 \end{pmatrix} \quad \text{and} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

Least squares for fitting a non-linear function

- Can write the least squares criterion in matrix notation

$$\arg \min_{\boldsymbol{\theta}} (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

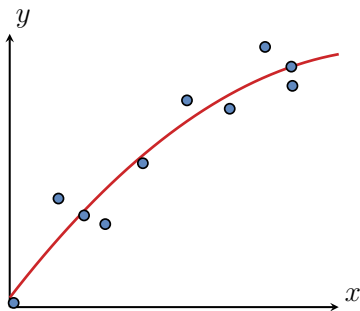
where

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \\ 1 & x_n & x_n^2 \end{pmatrix} \quad \text{and} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

- Have just solved this optimization problem and

$$\boldsymbol{\theta}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- Note have fit a non-linear function by applying a non-linear transformation to input and then solving a linear optimization problem.



Least squares for fitting a non-linear function

- Can write the least squares criterion in matrix notation

$$\arg \min_{\boldsymbol{\theta}} (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

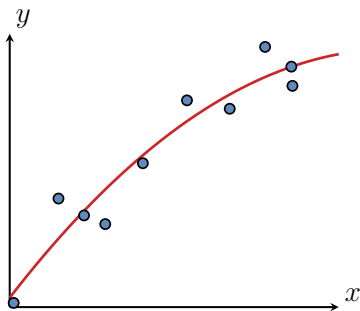
where

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{pmatrix} \quad \text{and} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

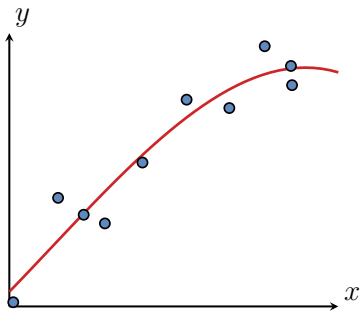
- Have just solved this optimization problem and

$$\boldsymbol{\theta}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- Note have fit a non-linear function by applying a **non-linear transformation to input** and then solving a **linear optimization** problem.



Least squares for fitting a non-linear function



- Same trick can be used to fit a cubic, just let

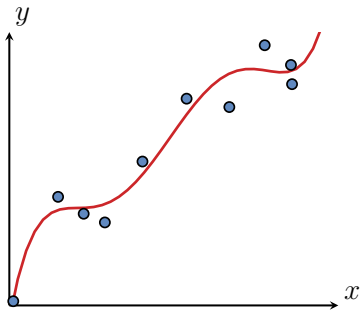
$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & x_n^3 \end{pmatrix}$$

- Can potentially fit a polynomial of degree p this way! Just let

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^p \\ 1 & x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{pmatrix}$$

- If I have n training points can I really fit a polynomial of any degree p with this method?

Least squares for fitting a non-linear function



- Same trick can be used to fit a cubic, just let

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & x_n^3 \end{pmatrix}$$

- Can potentially fit a polynomial of degree p this way! Just let

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^p \\ 1 & x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{pmatrix}$$

- If I have n training points can I really fit a polynomial of any degree p with this method?

Least squares for fitting a non-linear function

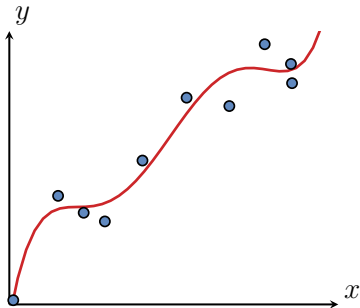
- Same trick can be used to fit a cubic, just let

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & x_n^3 \end{pmatrix}$$

- Can potentially fit a polynomial of degree p this way! Just let

$$X = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^p \\ 1 & x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^p \end{pmatrix}$$

- If I have n training points can I really fit a polynomial of any degree p with this method?



Need sufficient training points to fit polynomial of degree p

- Remember coefficients of polynomial are estimated by

$$\boldsymbol{\theta}^* = (X^T X)^{-1} X^T \mathbf{y}$$

- The size of $X^T X$ is $(p+1) \times (p+1)$ as X is $n \times (p+1)$
- What can we say about the rank of $X^T X$?

$$\text{rank}(X^T X) \leq \min(n, p+1)$$

$\implies X^T X$ is definitely singular when $n < p+1$.

$\implies (X^T X)^{-1}$ does not exist when $n < p+1$.

$\implies$ cannot find $\boldsymbol{\theta}^*$ with the above expression.

Need $\geq p + 1$ points to uniquely fit polynomial of degree p

- Remember: Size of X is $n \times (p + 1)$
- When $\text{rank}(X) \leq n < p + 1$
 - There exist multiple θ^* s.t.

$$X\theta^* - \mathbf{y} = \mathbf{0}$$

- Each θ^* has the form

$$\theta^* = X^\dagger \mathbf{y} + V\gamma, \quad \gamma \in \mathbb{R}^{p+1-\text{rank}(X)}$$

where

- * $X^\dagger$ is the pseudo-inverse of X .
($\text{rank}(X) = n$ then $X^\dagger = X^T(XX^T)^{-1}$)
- * each column of V is a basis vector for the nullspace of X .

Need $\geq p + 1$ points to uniquely fit polynomial of degree p

- Remember: Size of X is $n \times (p + 1)$
- When $\text{rank}(X) \leq n < p + 1$
 - There exist multiple θ^* s.t.

$$X\theta^* - \mathbf{y} = \mathbf{0}$$

- Each θ^* has the form

$$\theta^* = X^\dagger \mathbf{y} + V\gamma, \quad \gamma \in \mathbb{R}^{p+1-\text{rank}(X)}$$

where

- * $X^\dagger$ is the pseudo-inverse of X .
($\text{rank}(X) = n$ then $X^\dagger = X^T(XX^T)^{-1}$)
- * each column of V is a basis vector for the nullspace of X .

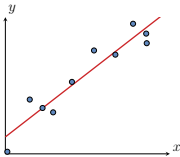
Back to machine learning...

- **Given:**

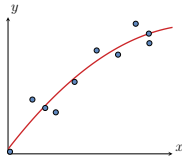
- n labelled training examples - $\{(x_1, y_1), \dots, (x_n, y_n)\}$.
- Lots of choices for form of the fitting function - all polynomials up to degree p .
- A method to estimate the parameters θ given a particular f - least squares estimation.

- **Problem:**

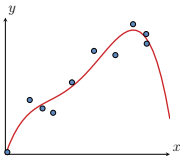
- How do I decide which function I should choose as my final predictor?



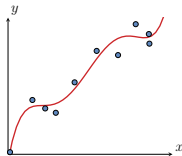
poly degree 1?



poly degree 2?



poly degree 4?



poly degree 5?

Option 1: Which polynomial degree?

- Choose the model that minimizes the training error.
- Assess performance for each possible function:

Calculate training error for each f

for $j = 1, \dots, p$

- Fit poly of degree j to training data to get θ_j^*
- Calculate the **training error**

$$\text{err}_j = \frac{1}{n} \sum_{i=1}^n (f_j(x_i, \theta_j^*) - y_i)^2$$

- Choose the optimal function with

$$j^* = \arg \min_{1 \leq j \leq p} \text{err}_j$$

Option 1: Which polynomial degree?

- Choose the model that minimizes the training error.
- Assess performance for each possible function:

Calculate training error for each f

for $j = 1, \dots, p$

- Fit polynomial f_j to training data and get θ_j^*
- Calculate the **training error**

$$\text{err}_j = \frac{1}{n} \sum_{i=1}^n (f_j(x_i, \theta_j^*) - y_i)^2$$

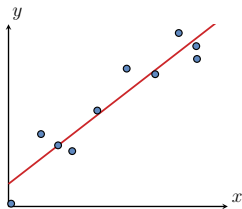
This is a terrible option.

- Choose the optimal function with

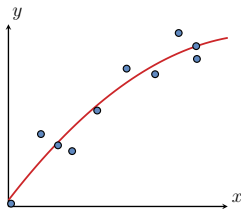
$$j^* = \arg \min_{1 \leq j \leq p} \text{err}_j$$

Don't measure performance with training error

Least squares fit of polynomials of different degrees to the training data.

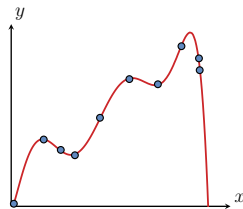


poly degree 1



poly degree 2

...

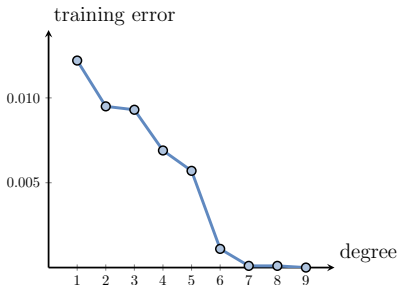


poly degree 8

- By increasing the complexity of the function can drive the training error to zero.

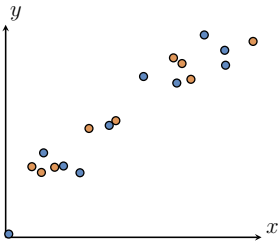
Don't measure performance with training error

Training error Vs polynomial degree for our toy problem.



- By increasing the complexity of the function can drive the training error to zero.

Instead generate a test set

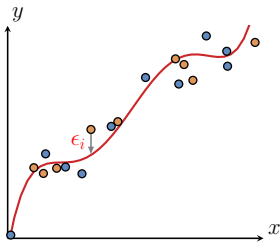


- Generate more labelled examples $\{(x_{n+i}, y_{n+i})\}_{i=1}^m$ (not used during training) called the **test set**.
- Define the **test error** for our problem as

$$\text{test err}_j = \frac{1}{m} \sum_{i=1}^m (f_j(x_{n+i}, \theta_j^*) - y_{n+i})^2$$

- **Test error** indicates how well the function **generalizes** to unseen samples.

Instead generate a test set



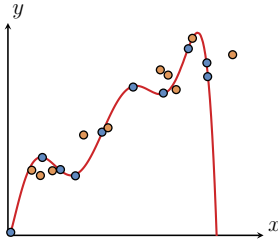
- Generate more labelled examples $\{(x_{n+i}, y_{n+i})\}_{i=1}^m$ (not used during training) called the **test set**.
- Define the **test error** for our problem as

$$\text{test err}_j = \frac{1}{m} \sum_{i=1}^m (f_j(x_{n+i}, \theta_j^*) - y_{n+i})^2$$

- **Test error** indicates how well the function **generalizes** to unseen samples.

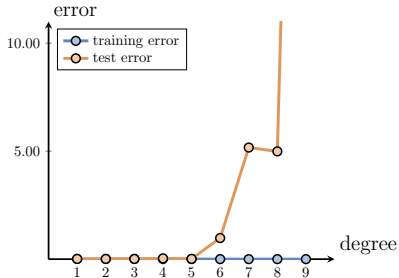
Low training error and high test error

Degree 8 poly has **training error** ≈ 0 and **test error** ≈ 5



- The **training** and **test error** can be very different.
- **Over-fitting** occurs when the training error is low but the test error is high.

Training and test error for our toy problem.



- Over-fitting occurs when the training error is low but the test error is high.
- The higher the capacity of your function $\implies$ more likely to over-fit.

Option 2: Which polynomial degree?

- **Criterion:** Choose the model that minimizes the test error.
- **Logistics:** Calculate **test error** for each possible function:

Calculate test error for each f

for $j = 1, \dots, p$

- Fit polynomial of degree j to the training data to get θ_j^*
- Calculate the **test error**

$$\text{test err}_j = \frac{1}{m} \sum_{i=1}^m (f_j(x_{n+i}, \theta_j^*) - y_{n+i})^2$$

- Choose the optimal function with

$$j^* = \arg \min_{1 \leq j \leq p} \text{test err}_j$$

Option 2: Which polynomial degree?

- **Criterion:** Choose the model that minimizes the test error.
- **Logistics:** Calculate **test error** for each possible function:

Calculate test error for each f

for $j = 1, \dots, p$

- Fit polynomial of degree j to the training data to get θ_j^*
- Calculate the **test error**

$$\text{test err}_j = \frac{1}{m} \sum_{i=1}^m (f_j(x_{n+i}, \theta_j^*) - y_{n+i})^2$$

- Choose the optimal function with

$$j^* = \arg \min_{1 \leq j \leq p} \text{test err}_j$$

What is the generalization ability of my final f_{j^*} ?

Pipeline for model selection & final performance measure

- Let $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^n$ represent all my labelled data.
- Partition $\mathcal{X}$ into 3 sets
 - the **training set** - $\mathcal{X}_{\text{train}}$,
 - the **validation set** - $\mathcal{X}_{\text{val}}$,
 - the **test set** - $\mathcal{X}_{\text{test}}$.
- Proceed as follows

1. **Model Selection** (measure performance on validation set)

for $j = 1, \dots, p$

- Fit polynomial of degree j using $\mathcal{X}_{\text{train}}$ to get θ_j^*
- Calculate the **validation error**

$$\text{val err}_j = \frac{1}{|\mathcal{X}_{\text{val}}|} \sum_{(x,y) \in \mathcal{X}_{\text{val}}} (f_j(x, \theta_j^*) - y)^2$$

Select a model: $j^* = \arg \min_{1 \leq j \leq p} \text{val err}_j$

Pipeline for model selection & final performance measure

- Let $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^n$ represent all my labelled data.
- Partition $\mathcal{X}$ into 3 sets
 - the **training set** - $\mathcal{X}_{\text{train}}$,
 - the **validation set** - $\mathcal{X}_{\text{val}}$,
 - the **test set** - $\mathcal{X}_{\text{test}}$.
- Proceed as follows

2. Final training of Selected Model (train using $\mathcal{X}_{\text{train}} \cup \mathcal{X}_{\text{val}}$)

- Fit polynomial of degree j^* given $\mathcal{X}_{\text{train}} \cup \mathcal{X}_{\text{val}}$ to get $\theta_{j^*}^*$

Pipeline for model selection & final performance measure

- Let $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^n$ represent all my labelled data.
- Partition $\mathcal{X}$ into 3 sets
 - the **training set** - $\mathcal{X}_{\text{train}}$,
 - the **validation set** - $\mathcal{X}_{\text{val}}$,
 - the **test set** - $\mathcal{X}_{\text{test}}$.
- Proceed as follows

3. Measure Performance (measure performance on test set)

- Assess performance of final regressor

$$\text{test err} = \frac{1}{|\mathcal{X}_{\text{test}}|} \sum_{(x,y) \in \mathcal{X}_{\text{test}}} (f_{j^*}(x, \theta_{j^*}^*) - y)^2$$

One central challenge of Machine Learning

- Ideally I want to
 1. Have an expressive prediction function f .
 2. Not over-fit during training.
- Simple model $\implies$ less likely to over-fit but can only accurately model simple relationships.
- High capacity model $\implies$ more likely to over-fit but can potentially accurately model complicated relationships.

- Ideally I want to
 1. Have an expressive prediction function f .
 2. Not over-fit during training.
- Simple model $\implies$ less likely to over-fit but can only accurately model simple relationships.
- High capacity model $\implies$ more likely to over-fit but can potentially accurately model complicated relationships.

- Ideally I want to
 1. Have an expressive prediction function f .
 2. Not over-fit during training.
- Simple model $\implies$ less likely to over-fit but can only accurately model simple relationships.
- High capacity model $\implies$ more likely to over-fit but can potentially accurately model complicated relationships.

- Ideally I want to
 1. Have an expressive prediction function f .
 2. Not over-fit during training.
- Simple model $\implies$ less likely to over-fit but can only accurately model simple relationships.
- High capacity model $\implies$ more likely to over-fit but can potentially accurately model complicated relationships.

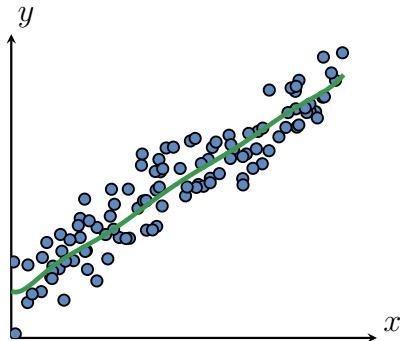
Can I have the best of both worlds??

Sometimes! **Regularization** -

Introduce extra constraints on your optimal model parameters.

Combat over-fitting: Use lots of labelled training data

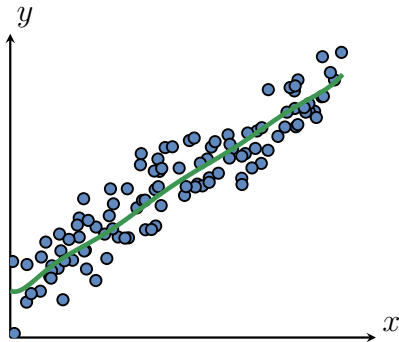
Least squares polynomial of degree 9 with 1000 training points.



Add many more labelled training points.

Combat over-fitting: Use lots of labelled training data

Least squares polynomial of degree 9 with 1000 training points.



Add many more labelled training points.

Unfortunately, this is often not an option....

Regularization: Add penalty term to the cost function

- Remember cost-function we've seen so far involves measuring how well θ fits the training data via a sum-of-squares measure:

$$L(\theta, \mathcal{X}) = (X\theta - \mathbf{y})^T (X\theta - \mathbf{y})$$

- Add an extra function, $R : \mathbb{R}^p \rightarrow \mathbb{R}$, **regularization** term, to the goodness-of-fit function (often termed the **loss function**)

$$C(\theta, \mathcal{X}) = L(\theta, \mathcal{X}) + \lambda R(\theta)$$

- Common choice for $R(\theta)$ is

$$R(\theta) = \|\theta\|^2 = \theta^T \theta$$

Regularization: Add penalty term to the cost function

- Add an extra function, $R : \mathbb{R}^p \rightarrow \mathbb{R}$, **regularization** term, to the goodness-of-fit function (often termed the **loss function**)

$$C(\boldsymbol{\theta}, \mathcal{X}) = L(\boldsymbol{\theta}, \mathcal{X}) + \lambda R(\boldsymbol{\theta})$$

- Regularization function should have *lower* scores for *simpler* models
 - $\implies$ its minimization should promote *simpler* models
 - $\implies$ discourage over-fitting.
- λ controls the trade-off between fitting the training data and *complexity* of the final model.

Example: **Ridge Regression** -

Regularization of the least squares regressor with $R(\boldsymbol{\theta}) = \boldsymbol{\theta}^T \boldsymbol{\theta}$.

Ridge regression for our toy problem

- Have the labelled training data $\mathcal{X} = \{(x_i, y_i)\}_{i=1}^n$
- Ridge regression solves this optimization problem

$$\arg \min_{\boldsymbol{\theta}, \theta_0} \left[(X\boldsymbol{\theta} + \theta_0\mathbf{1} - \mathbf{y})^T (X\boldsymbol{\theta} + \theta_0\mathbf{1} - \mathbf{y}) + \lambda \boldsymbol{\theta}^T \boldsymbol{\theta} \right]$$

where

$$X = \begin{pmatrix} x_1 & x_1^2 & \cdots & x_1^p \\ x_2 & x_2^2 & \cdots & x_2^p \\ \vdots & \vdots & \cdots & \vdots \\ x_n & x_n^2 & \cdots & x_n^p \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad \text{and} \quad \boldsymbol{\theta} = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{pmatrix}$$

- **Note:** Not putting regularization on the offset term.

- To make the solution of the optimization cleaner let

$$X_1 = X - \mathbf{1}^T \boldsymbol{\mu}$$

where

$$\mu_j = \frac{1}{n} \sum_{i=1}^n x_i^j, \quad \text{and} \quad \boldsymbol{\mu} = (\mu_1, \dots, \mu_p)^T$$

- This is called **centering the training data**.
- Important consequence

$$\mathbf{1}^T X_1 = \mathbf{0}^T$$

- The slightly re-jigged optimization problem is

$$\arg \min_{\boldsymbol{\theta}, \theta_0} \left[(X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y})^T (X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y}) + \lambda \boldsymbol{\theta}^T \boldsymbol{\theta} \right]$$

$$\arg \min_{\boldsymbol{\theta}, \theta_0} \left[(X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y})^T (X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y}) + \lambda \boldsymbol{\theta}^T \boldsymbol{\theta} \right]$$

- Can solve the optimization problem as before.
 1. Compute gradients of the cost function.
 2. Set expression for gradients to zero.
 3. Solve the resulting equation system.

$$\arg \min_{\boldsymbol{\theta}, \theta_0} \left[(X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y})^T (X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y}) + \lambda \boldsymbol{\theta}^T \boldsymbol{\theta} \right]$$

- Can solve the optimization problem as before.
 1. Compute gradients of the cost function.
 2. Set expression for gradients to zero.
 3. Solve the resulting equation system.

$$\arg \min_{\boldsymbol{\theta}, \theta_0} \left[(X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y})^T (X_1 \boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y}) + \lambda \boldsymbol{\theta}^T \boldsymbol{\theta} \right]$$

- Can solve the optimization problem as before.
 1. Compute gradients of the cost function.
 2. Set expression for gradients to zero.
 3. Solve the resulting equation system.
- **Solution** (must apply to a centered point)

$$\boldsymbol{\theta}^* = (X_1^T X_1 + \lambda I)^{-1} X_1^T \mathbf{y}, \quad \theta_0^* = \frac{1}{n} \sum_{i=1}^n y_i$$

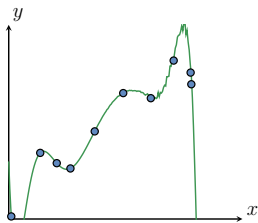
$$\arg \min_{\boldsymbol{\theta}, \theta_0} \left[(X\boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y})^T (X\boldsymbol{\theta} + \theta_0 \mathbf{1} - \mathbf{y}) + \lambda \boldsymbol{\theta}^T \boldsymbol{\theta} \right]$$

- Can solve the optimization problem as before.
 1. Compute gradients of the cost function.
 2. Set expression for gradients to zero.
 3. Solve the resulting equation system.
- **Solution** (for non-centered data)

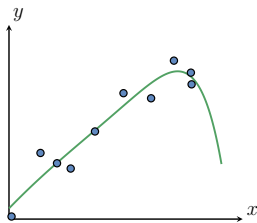
$$\boldsymbol{\theta}^* = (X_1^T X_1 + \lambda I)^{-1} X_1^T \mathbf{y}, \quad \theta_0^* = \frac{1}{n} \sum_{i=1}^n y_i - \boldsymbol{\mu}^T \boldsymbol{\theta}^*$$

Effect of λ on ridge regression solution

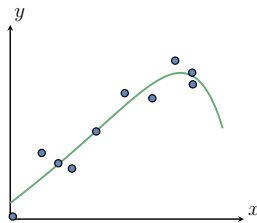
Estimated polynomial of degree 9 with different values of λ .



$\lambda = 0$



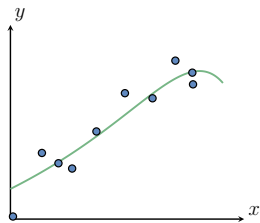
$\lambda = .001$



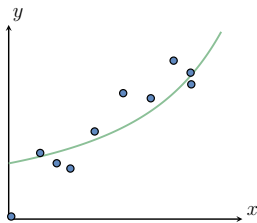
$\lambda = .01$

Effect of λ on ridge regression solution

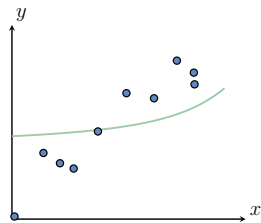
Estimated polynomial of degree 9 with different values of λ .



$\lambda = .1$



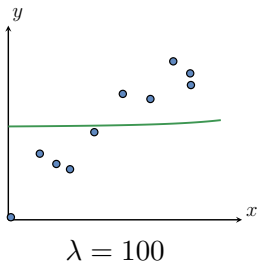
$\lambda = 1$



$\lambda = 10$

Effect of λ on ridge regression solution

Estimated polynomial of degree 9 with different values of λ .



Effect of λ on ridge regression solution

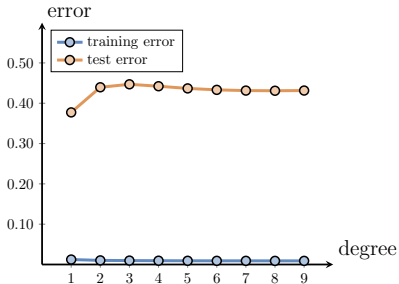
Coefficients of the degree 9 polynomial fit as λ varies.

λ	θ_0	θ_1	θ_2	$\dots$	θ_6	θ_7	θ_8	θ_9
0	1.215	-43.492	978.553	$\dots$	206857.077	-217202.970	126881.430	-31664.049
.001	0.877	1.585	-0.679	$\dots$	-0.043	-0.296	-0.516	-0.701
.010	0.916	1.175	0.254	$\dots$	-0.166	-0.201	-0.223	-0.235
.100	1.017	0.771	0.359	$\dots$	-0.098	-0.114	-0.119	-0.118
1.000	1.201	0.300	0.214	$\dots$	0.047	0.033	0.023	0.016
10.000	1.398	0.074	0.064	$\dots$	0.027	0.022	0.018	0.015
100.000	1.473	0.009	0.008	$\dots$	0.004	0.003	0.003	0.002

Effect of ridge-regression on training and test error



$\lambda = 0$



$\lambda = .001$

Training and test error Vs polynomial degree for our toy problem without and with regularization.

L_1 regularization is also popular and powerful...

- L_1 regularizer

$$R_{\text{lasso}}(\boldsymbol{\theta}) = \sum_{i=1}^p |\theta_i| = \|\boldsymbol{\theta}\|_1$$

- **Lasso Regression:** squared-error loss + L_1 regularization

$$\boldsymbol{\theta}_{\text{lasso}} = \arg \min \left[\sum_{i=1}^n (y_i - f(\mathbf{x}_i, \boldsymbol{\theta}) - \theta_0)^2 + \lambda \|\boldsymbol{\theta}\|_1 \right]$$

- L_1 regularizer

$$R_{\text{lasso}}(\boldsymbol{\theta}) = \sum_{i=1}^p |\theta_i| = \|\boldsymbol{\theta}\|_1$$

- **Lasso Regression:** squared-error loss + L_1 regularization

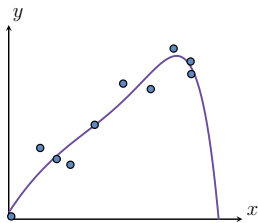
$$\boldsymbol{\theta}_{\text{lasso}} = \arg \min \left[\sum_{i=1}^n (y_i - f(\mathbf{x}_i, \boldsymbol{\theta}) - \theta_0)^2 + \lambda \|\boldsymbol{\theta}\|_1 \right]$$

Is there qualitative difference between $\boldsymbol{\theta}_{\text{lasso}}$ and $\boldsymbol{\theta}_{\text{ridge}}$?

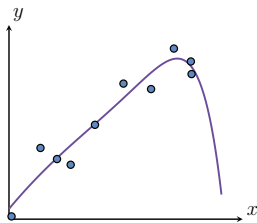
Yes!

Learnt lasso regressor as λ varies

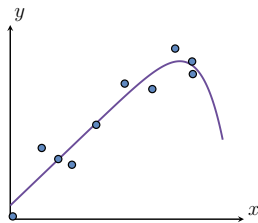
Estimated polynomial of degree 9 with different values of λ .



$\lambda = 6.5219e-04$



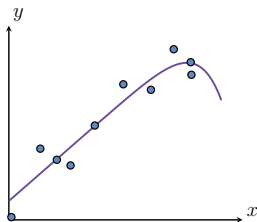
$\lambda = 0.0042$



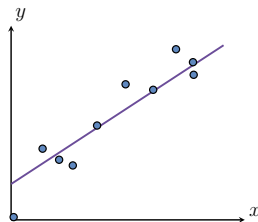
$\lambda = 0.0269$

Learnt lasso regressor as λ varies

Estimated polynomial of degree 9 with different values of λ .



$\lambda = 0.1732$



$\lambda = 1.1135$

Let's have a look at the coefficients

λ	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9
ϵ	9.3747	-63.8147	173.5530	-131.9416	-98.6952	56.0837	117.6399	54.3839	-122.3259
0.001	2.8568	-6.5747	8.2725	0	-3.4123	0	0	0	-1.0239
0.005	1.7327	-1.0029	0	1.2723	0	0	0	0	-1.9183
0.010	1.3763	0	0	0	0	0	0	0	-0.9639
0.050	1.3435	0	0	0	0	0	0	0	-0.8671
0.100	1.3026	0	0	0	0	0	0	0	-0.7462
1.000	0.9486	0	0	0	0	0	0	0	0

Let's have a look at the coefficients

λ	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9
ϵ	9.3747	-63.8147	173.5530	-131.9416	-98.6952	56.0837	117.6399	54.3839	-122.3259
0.001	2.8568	-6.5747	8.2725	0	-3.4123	0	0	0	-1.0239
0.005	1.7327	-1.0029	0	1.2723	0	0	0	0	-1.9183
0.010	1.3763	0	0	0	0	0	0	0	-0.9639
0.050	1.3435	0	0	0	0	0	0	0	-0.8671
0.100	1.3026	0	0	0	0	0	0	0	-0.7462
1.000	0.9486	0	0	0	0	0	0	0	0

- As λ increases, magnitude of non-zero coefficients shrink.
- And as λ increases many shrink to exactly zero.
- L_1 regularization promotes sparsity.
- (Iterative optimization algorithm $\implies$ don't reach minimum when $\lambda = 0$)

LASSO is great for feature selection.

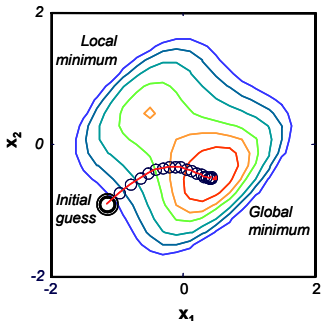
LASSO $\equiv$ Least absolute shrinkage and selection operator

Implicit Regularization via poor optimization:

gradient descent + early stopping

Gradient descent: Iterative function minimization

Gradient descent finds the minimum in an iterative fashion by moving in the direction of steepest descent.



Gradient Descent Minimization

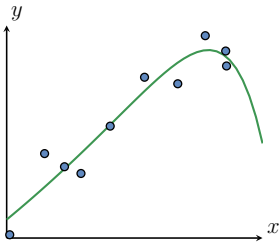
1. Start with an arbitrary solution $\mathbf{x}^{(0)}$.
2. Compute the gradient $\nabla_{\mathbf{x}} f(\mathbf{x}^{(k)})$.
3. Move in the direction of steepest descent:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \eta^{(k)} \nabla_{\mathbf{x}} f(\mathbf{x}^{(k)}).$$

where $\eta^{(k)}$ is the step size.

4. Go to 2 (until convergence).

Fit for toy example with gradient descent



- Fit polynomial of degree 10.
- $\eta = .001$
- # of update steps = 50,000
- **Early Stopping** $\equiv$ stop optimization before reaching optimum.

What generalizes from these slides to other machine learning methods?

What generalizes? Regression from $\mathbb{R}^d$

1. Want to learn a linear function $f : \mathbb{R}^d \times \mathbb{R}^{d+1} \rightarrow \mathbb{R}$ that is

$$f(\mathbf{x}, \boldsymbol{\theta}) = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{x} = (x_1, x_2, \dots, x_d)$ and $\boldsymbol{\theta} = (b, \mathbf{w}^T)^T$ and $\mathbf{w} \in \mathbb{R}^d$.

2. Have labelled data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with each $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$
3. Set

$$X = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \uparrow & \uparrow & \cdots & \uparrow \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_n \\ \downarrow & \downarrow & \cdots & \downarrow \end{pmatrix}^T$$

4. Can set up the least squares optimization exactly as before

$$\arg \min_{\boldsymbol{\theta}} (X\boldsymbol{\theta} - \mathbf{y})^T (X\boldsymbol{\theta} - \mathbf{y})$$

What generalizes? High-level approach to supervised learning

- **Task:** Learn a function $f : \mathbb{R}^d \times \mathbb{R}^p \rightarrow \mathbb{R}^k$
- **High-level solution:**
 1. Decide on how to represent f .
 2. Given labelled training and parametrization of f define an optimization problem which links f 's parameter setting to prediction quality on training data.
 3. Solve (or partially solve) the optimization problem to find a good parameter setting for f .
 4. Evaluate the found solution.

Every ML algorithm has three components:

- **Representation**
- **Optimization**
- **Evaluation**

Other take home message

- Machine learning methods can over-fit very easily especially for complicated f 's.
- You must be vigilant to avoid this.
- Regularization and data are your friends in this battle.

Over-fitting not the only peril when using Machine Learning

Dataset Bias - Do you see any sheep?

Microsoft Azure's computer vision API added the caption and tags.



A herd of sheep grazing on a lush green hillside
Tags: grazing, sheep, mountain, cattle, horse

Dataset Bias - I still don't see any sheep

Microsoft Azure's computer vision API added the caption and tags.



A close up of a hillside next to a rocky hill
Tags: hillside, grazing, sheep, giraffe, herd

Dataset Bias - But I see goats now

Microsoft Azure's computer vision API added the caption and tags.



Left: A man is holding a dog in his hand
Right: A woman is holding a dog in her hand

Image: @SouperSarah

- **Remember:** Computers do exactly what they are asked, so you have to be very careful in what you ask them to do!
- **Machine learning systems** will find the easiest path to solve the task you set them.
- Will exploit loopholes to find shortcuts usually because of:
 - dataset bias between training and test sets
 - poorly specified "loss" functions

- **Remember:** Computers do exactly what they are asked, so you have to be very careful in what you ask them to do!
- **Machine learning systems** will find the easiest path to solve the task you set them.
- Will exploit loopholes to find shortcuts usually because of:
 - dataset bias between training and test sets
 - poorly specified "loss" functions

- **Remember:** Computers do exactly what they are asked, so you have to be very careful in what you ask them to do!
- **Machine learning systems** will find the easiest path to solve the task you set them.
- Will exploit loopholes to find shortcuts usually because of:
 - dataset bias between training and test sets
 - poorly specified “loss” functions

- **Remember:** Computers do exactly what they are asked, so you have to be very careful in what you ask them to do!
- **Machine learning systems** will find the easiest path to solve the task you set them.
- Will exploit loopholes to find shortcuts usually because of:
 - dataset bias between training and test sets
 - poorly specified “loss” functions

Is your ML algorithm solving the problem you meant it to solve?

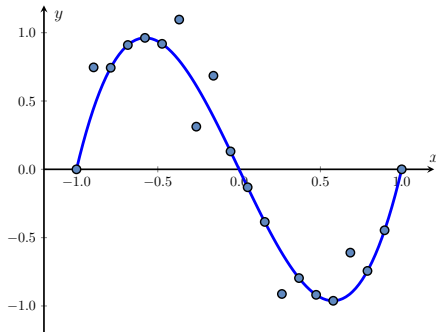
- **Remember:** Computers do exactly what they are asked, so you have to be very careful in what you ask them to do!
- **Machine learning systems** will find the easiest path to solve the task you set them.
- Will exploit loopholes to find shortcuts usually because of:
 - dataset bias between training and test sets
 - poorly specified “loss” functions

Is your ML algorithm solving the problem you meant it to solve?

Or just exploiting non-meaningful shortcuts.

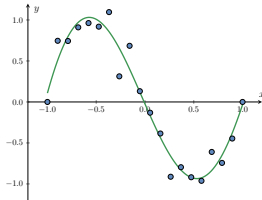
Postscript: Generalization for over-parametrized models not so well understood. (Slides based on [Deep Double Descent](#) blog post.)

Another toy example

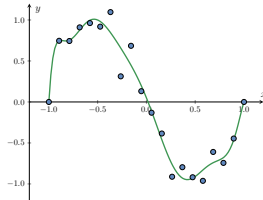


- Ground truth curve
- 20 training points with some noise

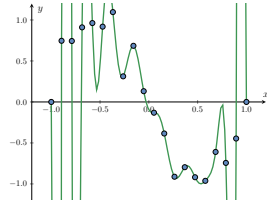
Fit polynomial of different degrees



$p = 5$



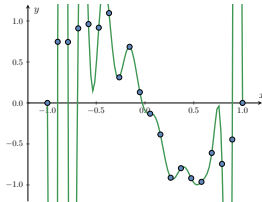
$p = 10$



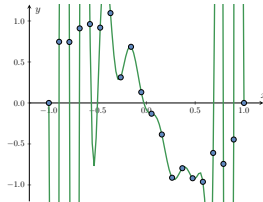
$p = 20$

- Calculate fit using pseudo-inverse of X & no regularization.
- Use standard basis $1, x, x^2, x^3, \dots, x^p$
- For $p \geq 20$ have very over-fit solutions.

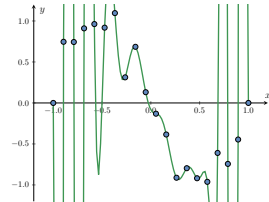
Fit polynomial of different degrees



$p = 20$



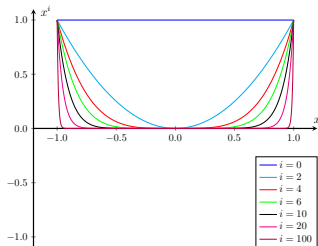
$p = 50$



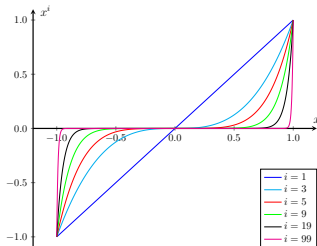
$p = 100$

- Calculate fit using pseudo-inverse of X & no regularization.
- Use standard basis $1, x, x^2, x^3, \dots, x^p$
- For $p \geq 20$ have very over-fit solutions.

Look at standard basis



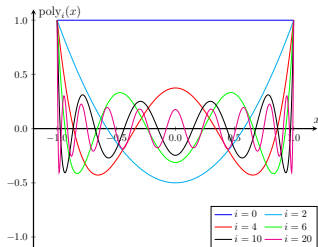
Even power basis fns



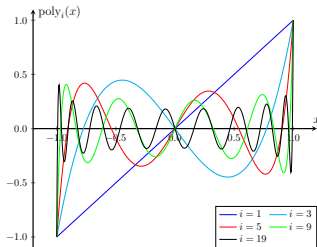
Odd power basis fns

Adding higher degree polynomials in this basis does not really help the representational power without very large coefficients.

Now let's consider the Legendre polynomial basis



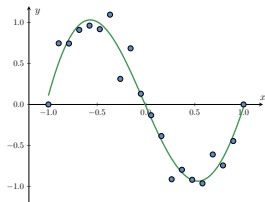
Even basis fns



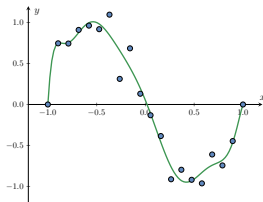
Odd basis fns

- Orthonormal basis.
- Can be derived from the standard basis using Gram-Schmidt orthonormalization.
- It can much more efficiently represent curves than the standard basis.

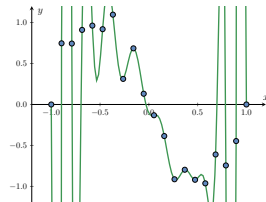
Legendre Polynomial fitting of curve



$p = 5$



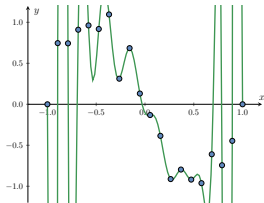
$p = 10$



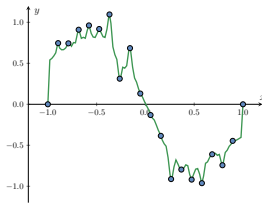
$p = 20$

So far everything is as expected.

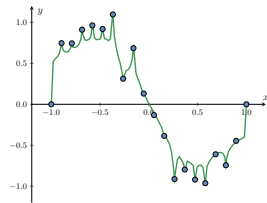
What if $p > 20$



$p = 20$



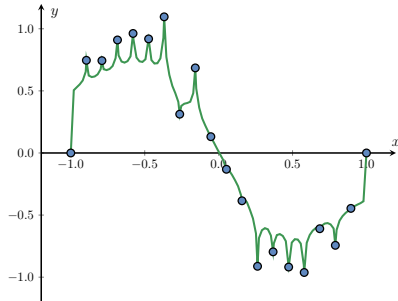
$p = 100$



$p = 200$

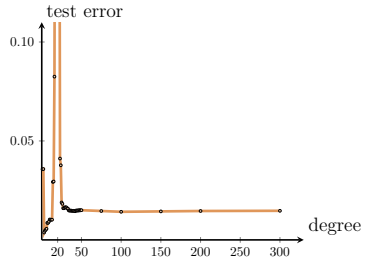
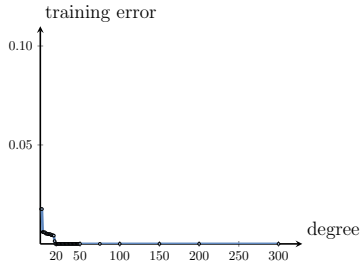
As p increases still interpolating but estimated curve is more reasonable.

What if $p \gg 20$

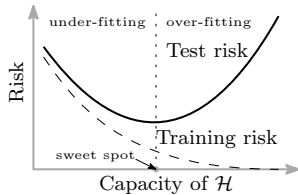


$p = 500$

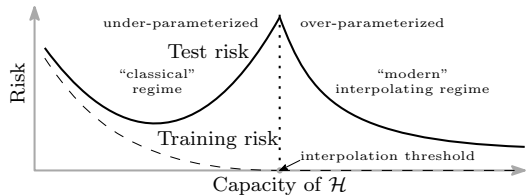
Test error plot as degree of poly increases



An example of Double Descent

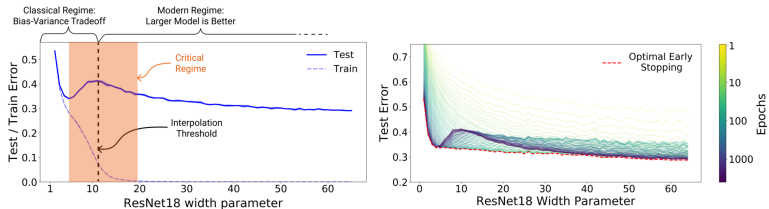


Classic bias-variance trade-off



Double descent risk curve

See same phenomenon in neural networks

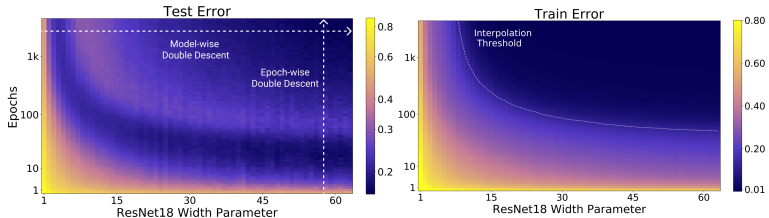


- ResNet18 network of varying widths fit to Cifar10 dataset with 15% label noise.
- **Right graph** Test error shown for varying train epochs.
- All models trained using Adam for 4K epochs.

DEEP DOUBLE DESCENT: WHERE BIGGER MODELS AND MORE DATA HURT

by P. Nakkiran et al, arXiv, 2019

Also have epochwise double descent



- **Left plot** Test error as a function of model size and train epochs.
- **Right plot** Train error of the corresponding models.
- ResNet18 network of varying widths fit to Cifar10 dataset with 15% label noise.
- All models trained using Adam for $\leq 4K$ epochs.

DEEP DOUBLE DESCENT: WHERE BIGGER MODELS AND MORE DATA HURT

by P. Nakkiran et al, arXiv, 2019

Postscript, Postscript: Deep learning & neural networks -
generalization not as well understood

The Mystery of Generalization in Deep Learning

- GoogLeNet, VGGNet,.... have many millions of parameters.
- Networks trained with (ignoring data augmentations)
training points $\ll$ # of parameters.
- But these networks still generalize well.
- What's going on??

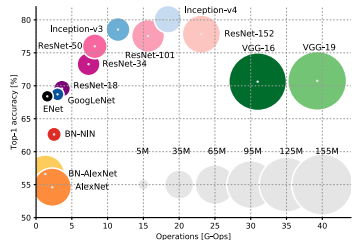


Fig credit: **An Analysis of Deep Neural Network Models for Practical Applications** by Canziani, Culurciello & Paszke.

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. If there was some unprocessed data that should have been added to the final page this extra page has been added to receive it. If you rerun the document (without altering it) this surplus page will go away, because $\text{\LaTeX}$ now knows how many pages to expect for this document.